

The Math: Region of convergence (ROC), mapping poles and zeros on the complex plane, and stability analysis.

The Project: Create an interactive Pole-Zero Placement Tool where moving poles/zeros dynamically updates the frequency response plot.

```
import numpy as np
import scipy.signal as signal
import matplotlib.pyplot as plt
from ipywidgets import interact, FloatSlider

def update_lti_system(zero_r=0.0, zero_theta=0.0, pole_r=0.0, pole_theta=0.0):
    """
    Dynamically recomputes and plots the Z-plane topology and
    corresponding frequency magnitude/phase response.
    """
    # 1. Convert Polar coordinates from sliders into Complex Conjugate Pairs
    # Zeros pair
    z1 = zero_r * np.exp(1j * np.radians(zero_theta))
    z2 = np.conj(z1)
    zeros = np.array([z1, z2]) if zero_theta != 0 else np.array([zero_r])

    # Poles pair
    p1 = pole_r * np.exp(1j * np.radians(pole_theta))
    p2 = np.conj(p1)
    poles = np.array([p1, p2]) if pole_theta != 0 else np.array([pole_r])

    # 2. Extract transfer function polynomial coefficients
    b, a = signal.zpk2tf(zeros, poles, k=1.0)

    # 3. Calculate Frequency Response over the Unit Circle
    w, h = signal.freqz(b, a, worN=1024)
    frequencies = w / np.pi # Normalized frequency (0 to 1, where 1 is Nyquist)

    # 4. Graphical Rendering Pipeline
    fig = plt.figure(figsize=(13, 5))

    # Plot A: The Z-Plane Complex Coordinate Grid
    ax1 = fig.add_subplot(1, 2, 1)
    # Draw Unit Circle boundary
    unit_circle = plt.Circle((0,0), 1.0, color='gray', fill=False, linestyle='--', linewidth=2)
    ax1.add_patch(unit_circle)

    # Plot locations
    ax1.scatter(np.real(zeros), np.imag(zeros), marker='o', s=100, edgecolors='blue', facecolors='white')
    ax1.scatter(np.real(poles), np.imag(poles), marker='x', s=100, color='red', linewidth=2)

    # Axis dressing
    ax1.axhline(0, color='black', alpha=0.3)
    ax1.axvline(0, color='black', alpha=0.3)
    ax1.set_xlim([-1.5, 1.5])
    ax1.set_ylim([-1.5, 1.5])
    ax1.set_title("Z-Plane Pole/Zero Topology")
    ax1.set_xlabel("Real Axis ( $\sigma$ )")
    ax1.set_ylabel("Imaginary Axis ( $j\omega$ )")
    ax1.grid(True, alpha=0.5)
    ax1.legend(loc="upper right")
    ax1.set_aspect('equal')

    # Plot B: The Frequency Magnitude Response Spectrum
    ax2 = fig.add_subplot(1, 2, 2)
    magnitude_db = 20 * np.log10(np.abs(h) + 1e-5) # 1e-5 safety offset avoids log(0)
```

```

ax2.plot(frequencies, magnitude_db, color='darkgreen', linewidth=2)
ax2.set_title("System Frequency Magnitude Response")
ax2.set_xlabel("Normalized Frequency ( $\times \pi$  rad/sample)")
ax2.set_ylabel("Gain (dB)")
ax2.set_ylim([-40, 40])
ax2.grid(True, alpha=0.5)

# Stability warning alert box
if pole_r >= 1.0:
    ax2.text(0.2, 0.5, "⚠ SYSTEM UNSTABLE\n(Poles outside unit circle)", color='white',
            weight='bold', fontsize=12, ha='center', va='center', transform=ax2.transAxes,
            bbox=dict(facecolor='red', alpha=0.9, boxstyle='round,pad=1'))

plt.tight_layout()
plt.show()

# Setup Dashboard UI Cockpit
interact(
    update_lti_system,
    zero_r      = FloatSlider(min=0.0, max=1.2, step=0.05, value=0.9, description='Zero Radius'),
    zero_theta  = FloatSlider(min=0.0, max=180.0, step=2.0, value=45.0, description='Zero Angle°'),
    pole_r      = FloatSlider(min=0.0, max=1.2, step=0.05, value=0.7, description='Pole Radius'),
    pole_theta  = FloatSlider(min=0.0, max=180.0, step=2.0, value=90.0, description='Pole Angle°'),
);

```

Zero Radius 0.35

Zero Angle° 90.00

Pole Radius 0.90

Pole Angle° 98.00



